

UNIVERSIDAD DE LAS FUERZAS ARMADAS



“ESPE”

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

INGENIERÍA EN ANÁLISIS Y DISEÑO DE SOFTWARE

CARRERA DE INGENIERÍA EN SOFTWARE



NOMBRE:

- Blacio Machuca Julio César
- Sigcha Palma Kenned Jhair
- Solano Sanchez Josue Vital
- Toapanta Jama Alexander Jhosue

NRC: 30724

FECHA: 28/04/2026

DOCENTE: Ing. Jenny Ruiz Robalino

1. Caso Base: Gestión de la Agenda de Clases (CUC-ADM-01)

El sistema debe permitir al administrador gestionar la agenda de clases de CrossFit. Cada clase tendrá la siguiente información mínima.

Dato	Tipo sugerido	Descripción	Validación mínima
Día y Hora	Fecha/Hora	Momento en que se impartirá la clase.	Obligatorio. No debe solaparse con otra clase existente.
Duración	Tiempo	Tiempo estimado de la sesión de entrenamiento.	Obligatorio.
Cupo Máximo	Entero	Número máximo de atletas que pueden reservar.	Obligatorio. No debe exceder la capacidad física del box.
Entrenador Asignado	Relación (ID)	Entrenador responsable de impartir la clase.	Obligatorio. Debe ser un usuario con rol "Entrenador" registrado y disponible en el sistema.

Requisitos Funcionales de nivel 0

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
RF-ADM-01.1	El sistema debe permitir al administrador crear una nueva clase, definiendo día, hora, duración y cupo máximo, y asignando un entrenador disponible.	Administrador	CUC-ADM-01 Administrar la agenda de clases

RF-ADM-01.2	El sistema debe permitir al administrador modificar los datos de una clase existente (horario, cupo, entrenador asignado).	Administrador	CUC-ADM-01 Administrar la agenda de clases
RF-ADM-01.3	El sistema debe permitir al administrador eliminar una clase existente, liberando el horario para futuras programaciones.	Administrador	CUC-ADM-01 Administrar la agenda de clases
RF-ADM-01.4	El sistema debe mostrar al administrador todas las clases programadas en una vista de agenda, detallando día, hora, entrenador y cupos libres.	Administrador	CUC-ADM-01 Administrar la agenda de clases
RF-ADM-01.5	El sistema debe validar que el entrenador asignado esté disponible y que el horario no se solape con otra clase existente antes de confirmar la creación o modificación.	Sistema	CUC-ADM-01 Administrar la agenda de clases

2. Caso Base: Gestión de Membresías y Pagos (CUC-ADM-02)

El sistema debe permitir al administrador gestionar las membresías y los pagos de los atletas. Cada membresía tendrá la siguiente información mínima.

Dato	Tipo sugerido	Descripción	Validación mínima
Tipo de Membresía	Texto	Nombre del plan (Básica, Premium, etc.).	Obligatorio.

Precio	Decimal	Costo monetario del plan de membresía.	Obligatorio. Debe ser numérico y mayor a 0.
Características	Texto largo	Beneficios y condiciones incluidas en el plan.	Obligatorio.
Estado de Pago	Booleano/Texto	Estado actual del pago del atleta (Pagado, Pendiente, Vencido).	Obligatorio. Se actualiza automáticamente según el registro de pagos.
Fecha de Vencimiento	Fecha	Fecha límite de vigencia de la membresía del atleta.	Obligatorio. Se calcula automáticamente al registrar un pago.
Historial de Pagos	Lista/Conjunto	Registro histórico de todas las transacciones realizadas por el atleta.	Automático.

Requisitos Funcionales de nivel 0

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
RF-ADM-02.1	El sistema debe permitir al administrador crear y configurar diferentes tipos de membresías con sus precios y características.	Administrador	CUC-ADM-02 Gestionar Membresías y Pagos
RF-ADM-02.2	El sistema debe permitir al administrador modificar los datos de un tipo de membresía existente (precio, características, duración).	Administrador	CUC-ADM-02 Gestionar Membresías y Pagos

RF-ADM-02.3	El sistema debe permitir al administrador eliminar un tipo de membresía que ya no se oferte, siempre que ningún atleta la tenga activa.	Administrador	CUC-ADM-02 Gestionar Membresías y Pagos
RF-ADM-02.4	El sistema debe mostrar al administrador un listado completo de todos los atletas con el estado actual de su membresía y fecha de vencimiento.	Administrador	CUC-ADM-02 Gestionar Membresías y Pagos
RF-ADM-02.5	El sistema debe permitir al administrador registrar un pago realizado por un atleta y actualizar automáticamente el estado de su membresía y fecha de vencimiento.	Administrador	CUC-ADM-02 Gestionar Membresías y Pagos
RF-ADM-02.6	El sistema debe permitir al administrador generar reportes financieros de ingresos por membresías y estados de morosidad en un período de tiempo determinado.	Administrador	CUC-ADM-02 Gestionar Membresías y Pagos

3. Caso Base: Seguimiento del Rendimiento de Atletas (CUC-ENT-01)

El sistema debe permitir al entrenador registrar y hacer seguimiento del rendimiento de los atletas en los entrenamientos (WODs). Cada registro de rendimiento tendrá la siguiente información mínima.

Dato	Tipo sugerido	Descripción	Validación mínima
Atleta	Relación (ID)	Atleta cuyo rendimiento se va a registrar.	Obligatorio. Debe ser un atleta registrado y con rutinas asignadas.
Sesión de Entrenamiento	Relación (ID)	Clase o WOD específico en el que participó el atleta.	Obligatorio.
Tiempo	Tiempo	Tiempo total empleado en completar el WOD.	Opcional, según el tipo de WOD.
Repeticiones	Entero	Cantidad de repeticiones completadas.	Opcional, según el tipo de ejercicio.
Peso Utilizado	Decimal	Carga de peso utilizada en el ejercicio.	Opcional, según el tipo de WOD.
Puntuación	Entero/Decimal	Puntuación total obtenida en el WOD.	Opcional.

Requisitos Funcionales de nivel 0

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
RF-ENT-01.1	El sistema debe permitir al entrenador seleccionar un atleta y registrar los resultados de un WOD específico (tiempo, repeticiones, peso, puntuación).	Entrenador	CUC-ENT-01 Seguimiento del rendimiento de atletas
RF-ENT-01.2	El sistema debe permitir al entrenador modificar un registro de rendimiento de una sesión anterior en caso de error o ajuste.	Entrenador	CUC-ENT-01 Seguimiento del rendimiento de atletas

RF-ENT-01.3	El sistema debe mostrar al entrenador el historial completo de rendimiento de un atleta seleccionado, permitiendo ver la evolución en el tiempo.	Entrenador	CUC-ENT-01 Seguimiento del rendimiento de atletas
RF-ENT-01.4	El sistema debe alertar al entrenador si no existen rutinas asignadas al atleta seleccionado antes de intentar registrar un rendimiento.	Sistema	CUC-ENT-01 Seguimiento del rendimiento de atletas
RF-ENT-01.5	El sistema debe validar que los datos de rendimiento ingresados sean coherentes y emitir un aviso si se detectan valores atípicos o inconsistentes con el historial del atleta.	Sistema	CUC-ENT-01 Seguimiento del rendimiento de atletas

4. Caso Base: Gestión de Reservas de Clases (CUC-ATL-01)

El sistema debe permitir al atleta gestionar sus reservas de clases. Cada reserva tendrá la siguiente información mínima.

Dato	Tipo sugerido	Descripción	Validación mínima
Clase Seleccionada	Relación (ID)	Clase o WOD que el atleta desea reservar.	Obligatorio. Debe ser una clase existente en la agenda semanal.
Horario	Fecha/Hora	Día y hora de la clase reservada.	Se obtiene automáticamente de la clase seleccionada.
Entrenador	Texto	Nombre del entrenador que imparte la clase.	Se obtiene automáticamente de la clase seleccionada.

Cupos Disponibles	Entero	Número de plazas libres en la clase seleccionada.	Se muestra automáticamente. La reserva es rechazada si es 0.
Estado de la Reserva	Texto	Estado actual (Confirmada/Cancelada)	Se actualiza según la acción del atleta.
Membresía del Atleta	Relación	Estado de la membresía del atleta que realiza la reserva.	Debe estar vigente para permitir la reserva.

Requisitos Funcionales de nivel 0

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
RF-ATL-01.1	El sistema debe permitir al atleta crear una nueva reserva, visualizando los horarios semanales disponibles con su entrenador y cupos libres, y seleccionando la clase deseada.	Atleta	CUC-ATL-01 Gestionar Reservas de Clases
RF-ATL-01.2	El sistema debe permitir al atleta eliminar (cancelar) una reserva de clase previamente confirmada, liberando el cupo para otro atleta.	Atleta	CUC-ATL-01 Gestionar Reservas de Clases
RF-ATL-01.3	El sistema debe permitir al atleta modificar una reserva existente, cancelando la actual y seleccionando una nueva clase disponible.	Atleta	CUC-ATL-01 Gestionar Reservas de Clases

RF-ATL-01.4	El sistema debe mostrar al atleta un listado de todas sus reservas activas (Mis Reservas) con el detalle del horario, entrenador y estado.	Atleta	CUC-ATL-01 Gestionar Reservas de Clases
RF-ATL-01.5	El sistema debe validar que la membresía del atleta esté vigente antes de permitir confirmar una nueva reserva.	Sistema	CUC-ATL-01 Gestionar Reservas de Clases
RF-ATL-01.6	El sistema debe validar que la clase seleccionada tenga cupos disponibles antes de permitir confirmar la reserva.	Sistema	CUC-ATL-01 Gestionar Reservas de Clases
RF-ATL-01.7	El sistema debe validar que el atleta no exceda el límite de reservas simultáneas permitidas y que esté dentro del plazo para cancelar una reserva.	Sistema	CUC-ATL-01 Gestionar Reservas de Clases
RF-ATL-01.8	El sistema debe enviar una notificación de confirmación al atleta tras realizar o cancelar una reserva exitosamente.	Sistema	CUC-ATL-01 Gestionar Reservas de Clases

Casos de uso

```
@startuml
left to right direction
skinparam packageStyle rectangle
skinparam actorStyle awesome
```

```
actor "Administrador" as Admin
```

```
rectangle "Sistema IRONCLAD BOX" {
```

```
    usecase "CUC-ADM-01\nAdministrar la agenda\nde clases" as UC1
```

```
    usecase "Crear nueva clase" as UC1_1
```

```
    usecase "Modificar clase\nexistente" as UC1_2
```

```
    usecase "Eliminar clase" as UC1_3
```

```
    usecase "Mostrar todas\nlas clases programadas" as UC1_4
```

```
    usecase "Validar disponibilidad\nde entrenador y horario" as UC1_5
```

```
    UC1_1 -[hidden]right- UC1_2
```

```
    UC1_2 -[hidden]right- UC1_3
```

```
}
```

```
Admin --> UC1
```

```
UC1 ..> UC1_1 : <<include>>
```

```
UC1 ..> UC1_2 : <<include>>
```

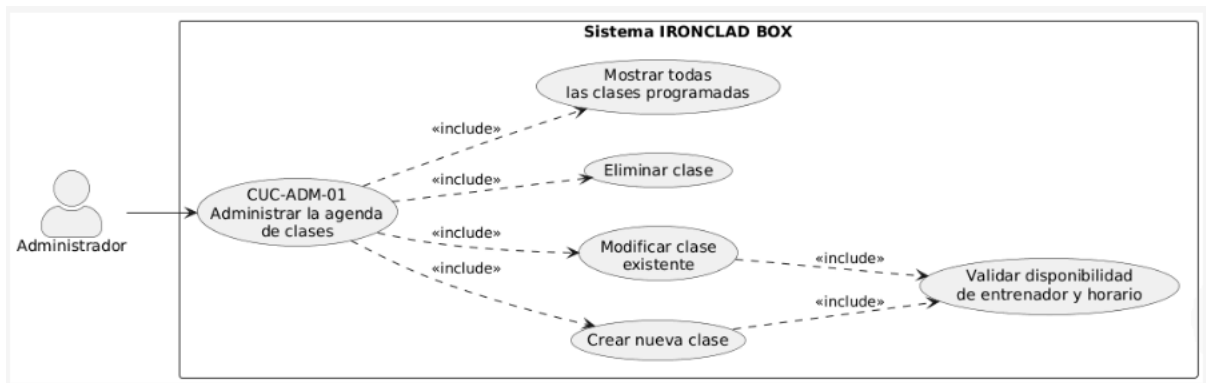
```
UC1 ..> UC1_3 : <<include>>
```

```
UC1 ..> UC1_4 : <<include>>
```

```
UC1_1 ..> UC1_5 : <<include>>
```

```
UC1_2 ..> UC1_5 : <<include>>
```

```
@enduml
```



```
@startuml
```

```
left to right direction
```

```
skinparam packageStyle rectangle
```

```
skinparam actorStyle awesome
```

```
actor "Administrador" as Admin
```

```
actor "Sistema de Pagos" as Pago <<System>>
```

```

rectangle "Sistema IRONCLAD BOX" {
  usecase "CUC-ADM-02\nGestionar Membresías\ny Pagos" as UC2

  usecase "Crear tipo\nde membresía" as UC2_1
  usecase "Modificar tipo\nde membresía" as UC2_2
  usecase "Eliminar tipo\nde membresía" as UC2_3
  usecase "Mostrar listado\nde atletas con estado\nde membresía" as UC2_4
  usecase "Registrar pago\nde atleta" as UC2_5
  usecase "Generar reportes\nfinancieros" as UC2_6
  usecase "Actualizar estado\nde membresía\nautomáticamente" as UC2_7
}

```

Admin --> UC2

Pago --> UC2_5

UC2 ..> UC2_1 : <<include>>

UC2 ..> UC2_2 : <<include>>

UC2 ..> UC2_3 : <<include>>

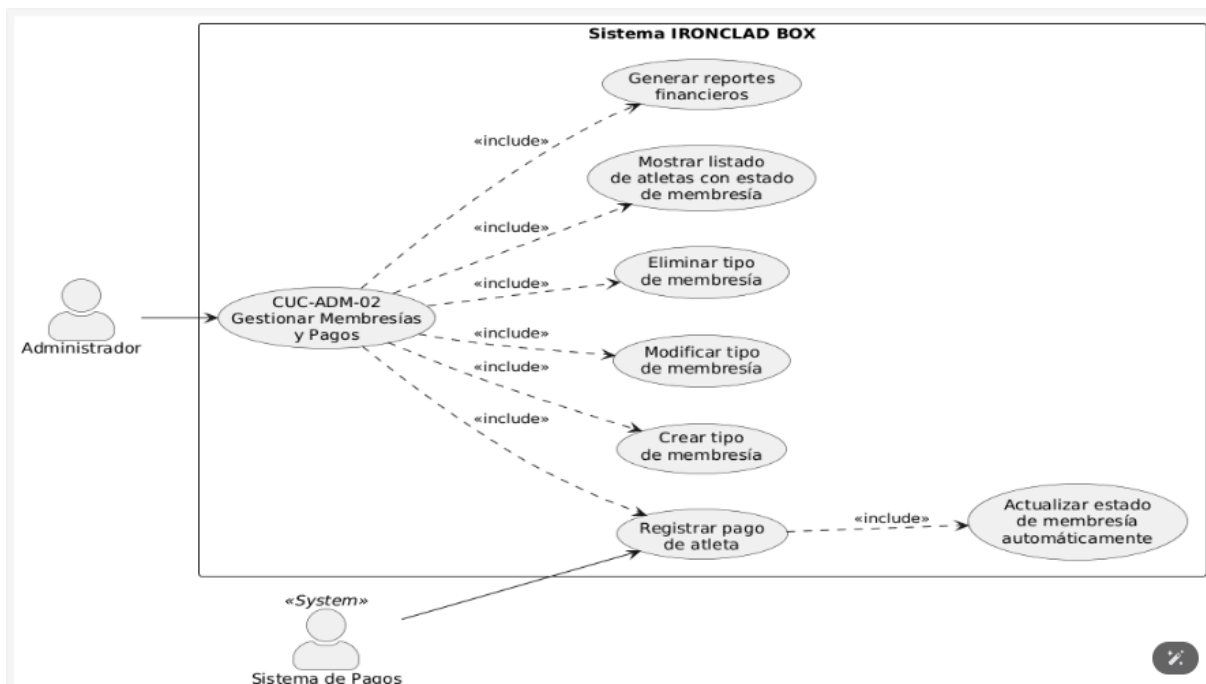
UC2 ..> UC2_4 : <<include>>

UC2 ..> UC2_5 : <<include>>

UC2 ..> UC2_6 : <<include>>

UC2_5 ..> UC2_7 : <<include>>

@enduml



```

@startuml
left to right direction
skinparam packageStyle rectangle
skinparam actorStyle awesome

actor "Entrenador" as Ent

rectangle "Sistema IRONCLAD BOX" {
    usecase "CUC-ENT-01\nSeguimiento del\nrendimiento de atletas" as UC_ENT1

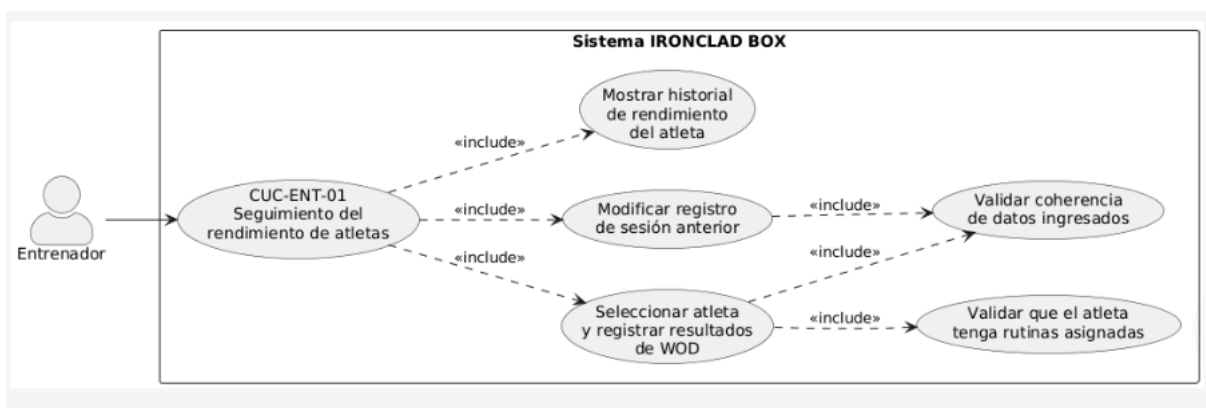
    usecase "Seleccionar atleta\ny registrar resultados\nde WOD" as ENT1_1
    usecase "Modificar registro\nde sesión anterior" as ENT1_2
    usecase "Mostrar historial\nde rendimiento\ndel atleta" as ENT1_3
    usecase "Validar que el atleta\ntenga rutinas asignadas" as ENT1_4
    usecase "Validar coherencia\nde datos ingresados" as ENT1_5
}

Ent --> UC_ENT1

UC_ENT1 ..> ENT1_1 : <<include>>
UC_ENT1 ..> ENT1_2 : <<include>>
UC_ENT1 ..> ENT1_3 : <<include>>
ENT1_1 ..> ENT1_4 : <<include>>
ENT1_1 ..> ENT1_5 : <<include>>
ENT1_2 ..> ENT1_5 : <<include>>

@enduml

```



```

@startuml
left to right direction
skinparam packageStyle rectangle

```

skinparam actorStyle awesome

actor "Atleta" as Atl

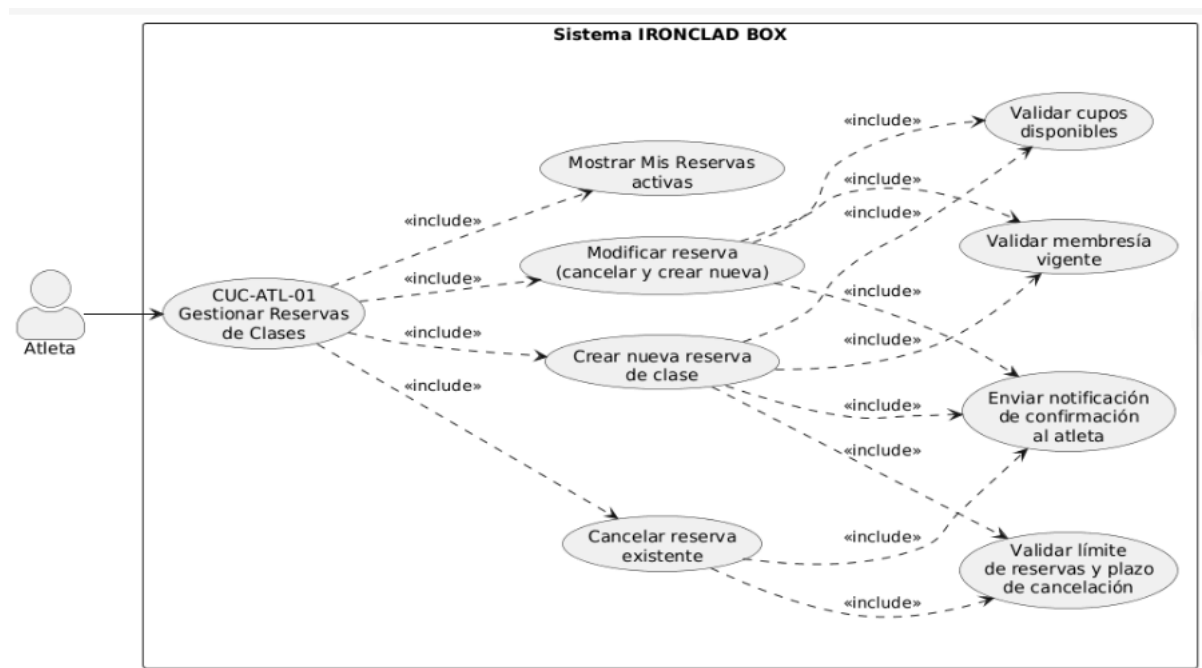
```
rectangle "Sistema IRONCLAD BOX" {
    usecase "CUC-ATL-01\nGestionar Reservas\nde Clases" as UC_ATL1

    usecase "Crear nueva reserva\nde clase" as ATL1_1
    usecase "Cancelar reserva\nexistente" as ATL1_2
    usecase "Modificar reserva\n(cancelar y crear nueva)" as ATL1_3
    usecase "Mostrar Mis Reservas\nactivas" as ATL1_4
    usecase "Validar membresía\nvigente" as ATL1_5
    usecase "Validar cupos\nDisponibles" as ATL1_6
    usecase "Validar límite\nde reservas y plazo\nde cancelación" as ATL1_7
    usecase "Enviar notificación\nde confirmación\nal atleta" as ATL1_8
}
```

Atl --> UC_ATL1

```
UC_ATL1 ..> ATL1_1 : <<include>>
UC_ATL1 ..> ATL1_2 : <<include>>
UC_ATL1 ..> ATL1_3 : <<include>>
UC_ATL1 ..> ATL1_4 : <<include>>
ATL1_1 ..> ATL1_5 : <<include>>
ATL1_1 ..> ATL1_6 : <<include>>
ATL1_1 ..> ATL1_7 : <<include>>
ATL1_1 ..> ATL1_8 : <<include>>
ATL1_2 ..> ATL1_7 : <<include>>
ATL1_2 ..> ATL1_8 : <<include>>
ATL1_3 ..> ATL1_5 : <<include>>
ATL1_3 ..> ATL1_6 : <<include>>
ATL1_3 ..> ATL1_8 : <<include>>
```

@enduml



Explicación breve de los casos de uso

Caso de uso	Actor	Precondición	Flujo principal resumido	Resultado esperado
CUC-ADM-01 Administrar la agenda de clases	Administrador	Admin autenticado. Entrenadores registrados.	Entrar al módulo de horarios → Crear/Modificar/Eliminar clase → Definir día, hora, duración y cupo máximo → Asignar entrenador disponible → Validar no solapamiento de horario → Confirmar y guardar.	Clase creada, modificada o eliminada. Horarios actualizados y disponibles para reservas de atletas.
CUC-ADM-02 Gestionar Membresías y Pagos	Administrador + Sistema de Pagos	Atletas registrados. Módulo de pagos configurado.	Acceder al módulo de membresías → Ver listado de atletas y estado → Crear/configurar tipo de membresía (nombre, precio, características) → Registrar pago de atleta → Actualizar estado y fecha de vencimiento automáticamente →	Membresías actualizadas. Reportes financieros de ingresos y morosidad generados.

			Generar reporte financiero.	
CUC-ENT-01 Seguimiento del rendimiento de atletas	Entrenador	Entrenador autenticado. Atleta con rutinas asignadas.	Acceder al módulo de Rendimiento → Seleccionar atleta y sesión de entrenamiento → Registrar resultados del WOD (tiempo, repeticiones, peso, puntuación) → Validar coherencia de datos → Guardar el registro.	Rendimiento del atleta registrado y disponible para análisis de evolución.
CUC-ATL-01 Gestionar Reservas de Clases	Atleta	Atleta con cuenta activa y membresía vigente. Clases disponibles.	Acceder al módulo de reservas → Ver horarios semanales disponibles (día, entrenador, cupos libres) → Seleccionar clase → Validar membresía vigente y cupos disponibles → Confirmar reserva → Recibir notificación de confirmación.	Reserva confirmada. Clase visible en "Mis Reservas" del atleta.

Puente hacia clases de análisis: frontera, control y entidad

1. CUC-ADM-01: Administrar la Agenda de Clases

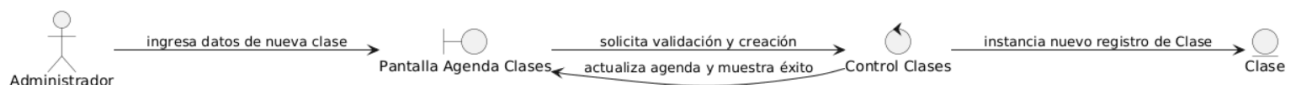
Tipo de clase	Propósito en análisis	Ejemplo para IronClad Box	Sintaxis PlantUML
Frontera / Boundary	Pantalla de gestión de horarios y clases.	PantallaAgendaClases	boundary "Pantalla Agenda Clases" as UI
Control / Control	Coordina creación, modificación y validación de clases.	ControlClases	control "Control Clases" as Ctrl
Entidad / Entity	Clase o WOD programado que el sistema conserva.	Clase	entity "Clase" as Est

```

@startuml
left to right direction
actor "Administrador" as Admin
boundary "Pantalla Agenda Clases" as UI
control "Control Clases" as Ctrl
entity "Clase" as EntClase

Admin --> UI: ingresa datos de nueva clase
UI --> Ctrl: solicita validación y creación
Ctrl --> EntClase: instancia nuevo registro de Clase
Ctrl --> UI: actualiza agenda y muestra éxito
@enduml

```



2. CUC-ADM-02: Gestionar Membresías y Pagos

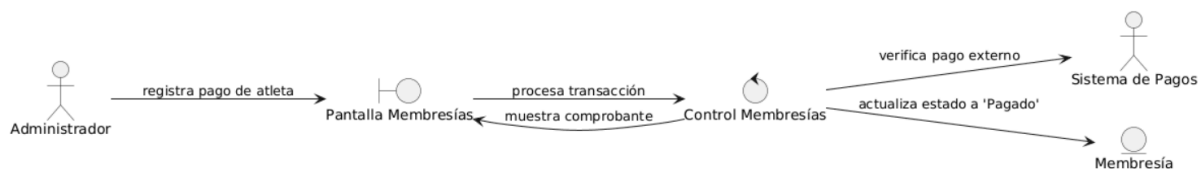
Tipo de clase	Propósito en análisis	Ejemplo para IronClad Box	Sintaxis PlantUML
Frontera / Boundary	Pantalla de gestión de membresías y pagos.	PantallaMembresias	boundary "Pantalla Membresías" as UI
Control / Control	Coordina lógica de membresías, pagos y reportes.	ControlMembresias	control "Control Membresías" as Ctrl
Entidad / Entity	Tipo de membresía y estado de pago del atleta.	Membresia	entity "Membresía" as Est

```

@startuml
left to right direction
actor "Administrador" as Admin
actor "Sistema de Pagos" as SP
boundary "Pantalla Membresías" as UI
control "Control Membresías" as Ctrl
entity "Membresía" as EntMem

Admin --> UI: registra pago de atleta
UI --> Ctrl: procesa transacción
Ctrl --> SP: verifica pago externo
Ctrl --> EntMem: actualiza estado a 'Pagado'
Ctrl --> UI: muestra comprobante
@enduml

```

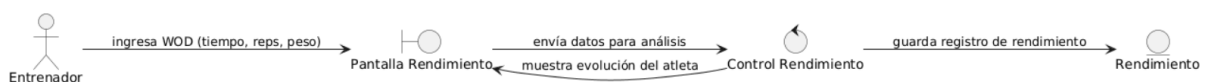
3. CUC-ENT-01: Seguimiento del Rendimiento de Atletas

Tipo de clase	Propósito en análisis	Ejemplo para IronClad Box	Sintaxis PlantUML
Frontera / Boundary	Pantalla de seguimiento de rendimiento de atletas.	PantallaRendimiento	boundary "Pantalla Rendimiento" as UI
Control / Control	Coordina registro y validación de resultados de WOD.	ControlRendimiento	control "Control Rendimiento" as Ctrl
Entidad / Entity	Registro de rendimiento de un WOD del atleta.	Rendimiento	entity "Rendimiento" as Est

```

@startuml
left to right direction
actor "Entrenador" as Ent
boundary "Pantalla Rendimiento" as UI
control "Control Rendimiento" as Ctrl
entity "Rendimiento" as EntRend

Ent --> UI: ingresa WOD (tiempo, reps, peso)
UI --> Ctrl: envía datos para análisis
Ctrl --> EntRend: guarda registro de rendimiento
Ctrl --> UI: muestra evolución del atleta
@enduml
  
```



4. CUC-ATL-01: Gestionar Reservas de Clases

Tipo de clase	Propósito en análisis	Ejemplo para IronClad Box	Sintaxis PlantUML
Frontera / Boundary	Pantalla de reservas de clases.	PantallaReservas	boundary "Pantalla Reservas" as UI

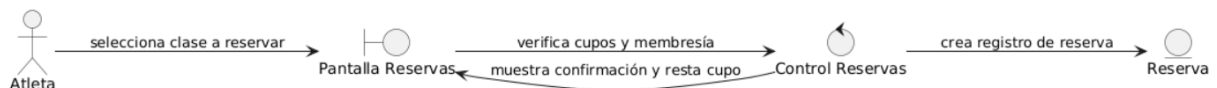
Control / Control	Coordina lógica de reserva, cancelación y validación de cupos.	ControlReservas	control "Control Reservas" as Ctrl
Entidad / Entity	Reserva de clase del atleta.	Reserva	entity "Reserva" as Est

```

@startuml
left to right direction
actor "Atleta" as Atl
boundary "Pantalla Reservas" as UI
control "Control Reservas" as Ctrl
entity "Reserva" as EntRes

Atl --> UI: selecciona clase a reservar
UI --> Ctrl: verifica cupos y membresía
Ctrl --> EntRes: crea registro de reserva
Ctrl --> UI: muestra confirmación y resta cupo
@enduml

```



Matriz de trazabilidad RF-CU

RF	Caso de uso	Actor	Prioridad	Criterio de aceptación	Evidencia PlantUML
Crear nueva clase	CUC-ADM-01 Administrar la agenda de clases	Administrador	Alta	El sistema guarda una clase con día, hora, duración, cupo máximo y entrenador asignado, validando no solapamiento de horarios.	UC1

Modificar clase existente	CUC-ADM-01 Administrar la agenda de clases	Administrador	Alta	El sistema modifica los datos de una clase existente luego de validar disponibilidad del entrenador.	UC1
Eliminar clase	CUC-ADM-01 Administrar la agenda de clases	Administrador	Media	El sistema elimina una clase y libera el horario para futuras programaciones.	UC1
Mostrar todas las clases programadas	CUC-ADM-01 Administrar la agenda de clases	Administrador	Alta	El sistema muestra todas las clases programadas en una vista de agenda semanal con día, hora, entrenador y cupos libres.	UC1
Validar disponibilidad de entrenador y horario	CUC-ADM-01 Administrar la agenda de clases	Sistema	Alta	El sistema impide guardar clases con horario solapado o entrenador no disponible.	UC1
Crear tipo de membresía	CUC-ADM-02 Gestionar Membresías y Pagos	Administrador	Alta	El sistema guarda un nuevo tipo de membresía con nombre, precio y características.	UC2
Modificar tipo de membresía	CUC-ADM-02 Gestionar Membresías y Pagos	Administrador	Media	El sistema modifica precio, características o duración de una membresía existente.	UC2

Eliminar tipo de membresía	CUC-ADM-02 Gestionar Membresías y Pagos	Administrador	Baja	El sistema elimina un tipo de membresía que no esté asignada a ningún atleta activo.	UC2
Mostrar listado de atletas con estado de membresía	CUC-ADM-02 Gestionar Membresías y Pagos	Administrador	Alta	El sistema muestra listado de atletas con estado de membresía y fecha de vencimiento.	UC2
Registrar pago de atleta	CUC-ADM-02 Gestionar Membresías y Pagos	Administrador + Sistema de Pagos	Alta	El sistema registra un pago y actualiza automáticamente el estado de membresía del atleta.	UC2
Generar reportes financieros	CUC-ADM-02 Gestionar Membresías y Pagos	Administrador	Media	El sistema genera reportes financieros de ingresos y morosidad en un período seleccionado.	UC2
Seleccionar atleta y registrar resultados de WOD	CUC-ENT-01 Seguimiento del rendimiento de atletas	Entrenador	Alta	El sistema guarda resultados de WOD (tiempo, repeticiones, peso, puntuación) para un atleta seleccionado con rutinas asignadas.	UC3

Modificar registro de sesión anterior	CUC-ENT-01 Seguimiento del rendimiento de atletas	Entrenador	Media	El sistema modifica un registro de rendimiento de una sesión anterior en caso de error o ajuste.	UC3
Mostrar historial de rendimiento del atleta	CUC-ENT-01 Seguimiento del rendimiento de atletas	Entrenador	Alta	El sistema muestra historial completo de rendimiento de un atleta con evolución en el tiempo.	UC3
Validar que el atleta tenga rutinas asignadas	CUC-ENT-01 Seguimiento del rendimiento de atletas	Sistema	Alta	El sistema alerta si el atleta seleccionado no tiene rutinas asignadas antes de registrar rendimiento.	UC3
Validar coherencia de datos ingresados	CUC-ENT-01 Seguimiento del rendimiento de atletas	Sistema	Media	El sistema emite aviso si los datos de rendimiento son inconsistentes con el historial del atleta.	UC3
Crear nueva reserva de clase	CUC-ATL-01 Gestionar Reservas de Clases	Atleta	Alta	El sistema crea una reserva tras seleccionar clase con horario, entrenador y cupos libres disponibles.	UC4
Cancelar reserva existente	CUC-ATL-01 Gestionar Reservas de Clases	Atleta	Alta	El sistema cancela una reserva existente liberando el cupo para otro atleta.	UC4

Modificar reserva	CUC-ATL-01 Gestionar Reservas de Clases	Atleta	Media	El sistema modifica una reserva cancelando la actual y creando una nueva clase.	UC4
Mostrar Mis Reservas activas	CUC-ATL-01 Gestionar Reservas de Clases	Atleta	Alta	El sistema muestra listado de todas las reservas activas del atleta con horario, entrenador y estado.	UC4
Validar membresía vigente	CUC-ATL-01 Gestionar Reservas de Clases	Sistema	Alta	El sistema impide confirmar reserva si la membresía del atleta no está vigente.	UC4
Validar cupos disponibles	CUC-ATL-01 Gestionar Reservas de Clases	Sistema	Alta	El sistema impide confirmar reserva si la clase seleccionada no tiene cupos disponibles.	UC4
Validar límite de reservas y plazo de cancelación	CUC-ATL-01 Gestionar Reservas de Clases	Sistema	Media	El sistema valida que el atleta no exceda el límite de reservas simultáneas y que esté dentro del plazo para cancelar.	UC4
Enviar notificación de confirmación al atleta	CUC-ATL-01 Gestionar Reservas de Clases	Sistema	Alta	El sistema envía notificación de confirmación al atleta tras realizar o cancelar una	UC4

				reserva exitosamente.	
--	--	--	--	--------------------------	--

Conclusión técnica

Se completó el análisis de Nivel 0 del sistema IRONCLAD BOX, abarcando 10 casos de uso distribuidos en tres módulos (Administrador, Entrenador y Atleta), de los cuales se profundizó en 4 casos base prioritarios: administración de agenda de clases (CUC-ADM-01), gestión de membresías y pagos (CUC-ADM-02), seguimiento de rendimiento de atletas (CUC-ENT-01) y gestión de reservas de clases (CUC-ATL-01). Para cada uno se generaron diagramas de casos de uso en PlantUML, especificación breve con precondiciones y flujos, identificación de clases de análisis (Frontera, Control y Entidad), y una matriz de trazabilidad que vincula 24 requisitos funcionales con sus respectivos casos de uso, actores, prioridades y criterios de aceptación.

El análisis confirma que el SRS sigue el estándar IEEE 830, con una arquitectura tecnológica definida (HTML, CSS, JavaScript, PHP y MySQL), roles de usuario claramente delimitados y dos actores externos identificados (Sistema de Pagos y APIs de Salud) que introducen dependencias a gestionar. Las entidades transversales detectadas —Clase, Membresía, Rendimiento y Reserva— serán la base para el modelo de datos relacional, mientras que las validaciones automáticas asignadas al actor "Sistema" quedaron correctamente modeladas mediante relaciones <<include>> en los diagramas.